3 The Linux System: Core Concepts and Components

1 What is Linux?

Linux is an open-source, Unix-like operating system kernel created by **Linus Torvalds** in 1991. Paired with open-source utilities and software (often referred to as GNU/Linux), it forms a complete operating system capable of managing system hardware, memory, processes, and execution environments.

Unlike proprietary operating systems such as Microsoft Windows or macOS, Linux is developed under the **GNU General Public License (GPL)**. This licensing allows anyone to inspect, modify, and redistribute its source code freely.

Linux is typically distributed as part of a **Linux Distribution (Distro)**, which packages the Linux kernel together with system utilities, package managers, user interfaces, and default applications.

Popular Distributions:

- **Enterprise & Server:** Red Hat Enterprise Linux (RHEL), Debian, Ubuntu Server, Rocky Linux / AlmaLinux.
- **Desktop & General Use:** Ubuntu, Linux Mint, Fedora, Arch Linux, Manjaro.
- **Specialized & Security:** Kali Linux (Penetration Testing), Alpine Linux (Minimal Containers).



Figure 6: Linux distributions

1.1 Global Usage & Key Statistics

While Linux holds a modest share of the traditional desktop consumer market, it dominates server, cloud, and embedded infrastructure worldwide:

- **Supercomputing: 100%** of the world's top 500 fastest supercomputers run on Linux.
- **Global Desktop Share:** Linux holds roughly **3.09% to 4.54%** of the global desktop operating system market, according to recent 2026 data from sources like Statcounter Global Stats and Wikipedia.
- **Public Cloud & Web Servers:** Over **90%** of public cloud workloads (AWS, Google Cloud, Microsoft Azure) and top web servers operate on Linux distributions.
- **Mobile Ecosystem: Android**, the world's most widely used mobile operating system, is built on top of a modified Linux kernel.
- **Containers & DevOps:** Nearly **100%** of containerized infrastructure (Docker, Kubernetes) relies on Linux kernel primitives (e.g., `cgroups` and namespaces).

- **Top Distributions:** Ubuntu leads general desktop and developer adoption at roughly **33.9%** of distributions, followed by Debian and Linux Mint.

1.2 Linux Distributions

Linux distributions (often called **distros**) are complete operating systems built on top of the **Linux Kernel**, bundled with user applications, software repositories, and system libraries.

Because Linux is **open-source**, anyone can run, study, modify, and redistribute its source code. This sets Linux apart from proprietary operating systems like UNIX, Microsoft Windows, and macOS, which are closed-source and far less customizable.

Major Linux Distribution Families:

1. **Debian Family:** Known for stability and software quality.
    - *Derived Distros:***Ubuntu**,**Knoppix**,**Chrome OS**,**Mepis**,**Linux Mint**,**Damn Small Linux**.
2. **Red Hat Family:** Focused on enterprise and workstation environments.
    - *Derived Distros:***Fedora**,**CentOS**,**Mandriva**,**Mageia**,**PCLinuxOS**.
3. **Slackware Family:** One of the oldest active Linux distributions, emphasizing simplicity and minimalism.
    - *Derived Distros:***SUSE**.
4. **Other Independent Distributions:**
    - **Gentoo**,**Arch Linux**,**Puppy Linux**.

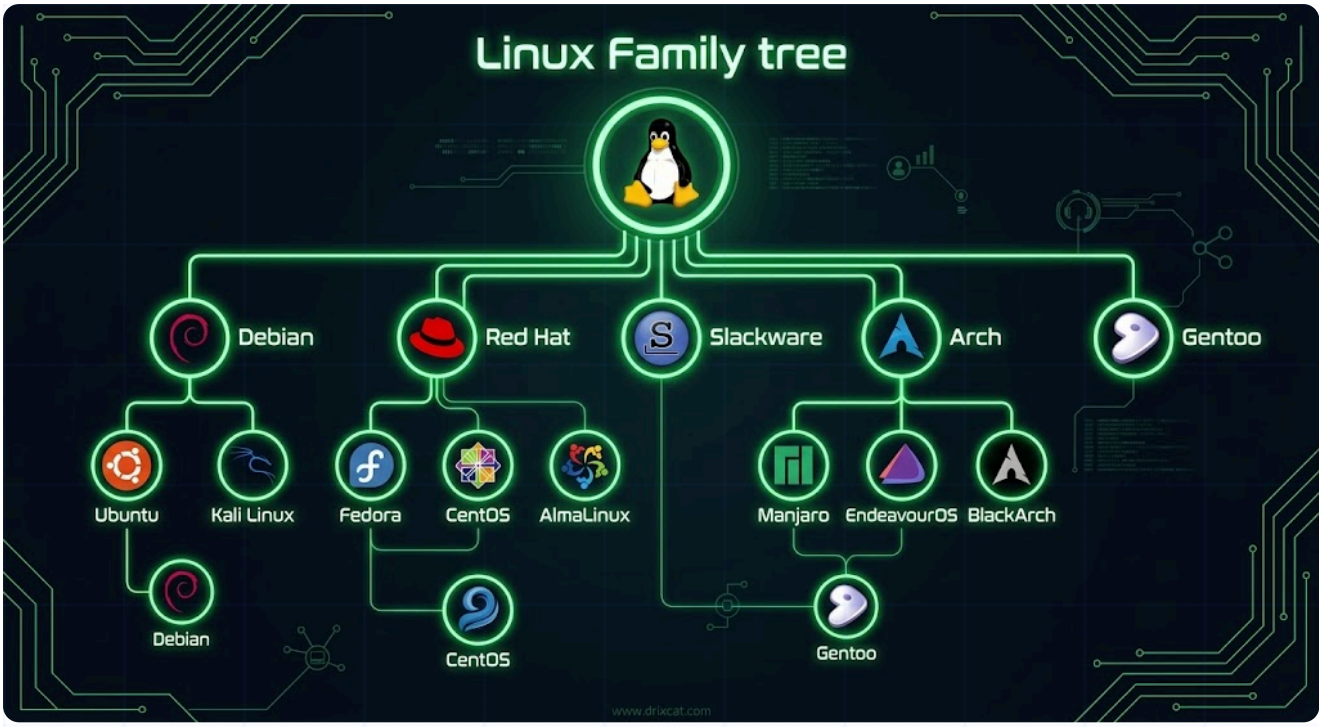


Figure 7: Linux Family Tree

1.3 Primary Advantages of Linux

- **Open-Source & Cost-Effective:** Free to use, deploy, and customize without licensing fees.
- **High Stability & Reliability:** Capable of running for years without needing a reboot, making it ideal for critical production infrastructure.
- **Granular Security:** Enforces strict multi-user access controls, privilege isolation (User Space vs. Kernel Space / Ring 3 vs. Ring 0), and mandatory access frameworks (SELinux, AppArmor).
- **Flexibility & Customization:** Highly modular—can be tailored from a minimal 5 MB headless container environment to a full desktop workstation.
- **Resource Efficiency:** Runs efficiently on legacy or resource-constrained hardware as well as high-performance servers.

1.4 Potential Challenges

- **Software Compatibility:** Certain proprietary desktop applications (e.g., Adobe Creative Cloud, native Microsoft Office) do not offer native Linux versions.
- **Hardware Driver Support:** While broad, some niche hardware or modern proprietary peripherals may require manual configuration or open-source driver workarounds.
- **Learning Curve:** Managing Linux effectively requires comfort with command-line interfaces (CLI), file permission models, and shell usage.

1.5 The Philosophy That "Everything Is A File"

In Linux, the philosophy that "everything is a file" is a fundamental concept. This concept means that in Linux, various system resources and components are represented as files or directories in the file system. This includes not only regular files but also devices, sockets, directories, and even processes.

- **Devices**: Hardware devices are represented as files in the /dev directory.
- **Processes**: Information about running processes can be accessed through files in the /proc directory.
- **Network communication**: Sockets are represented as files in the file system.
- **Configuration**: System configuration files are stored as text files in various directories.

2 Linux from the beginning - History and Evolution



2.1 The First Stage (1964–1984)

- In 1965, an agreement was formed among three large specialized companies to create an operating system that would serve researchers and other users:
  - **General Electric**, which acted as the funder for the project.
  - **Bell Laboratories**, which acted as a research institute.
  - **MIT**, which acted as an educational institute.
- In 1968, the first operating system named **Multics** was released. This system supported **Multitasking** and **Multiuser** capabilities, providing a **Time Sharing Operating System**. Despite the success of the project, ambitions were larger than the available capabilities, leading both General Electric and MIT to withdraw.

- Bell continued developing Multics under the leadership of developer Ken Thompson until 1970, when a new operating system named **Unics** was released, written in Assembly Language. The development of Unics continued until 1971, when its name was changed to **Unix**, marking a major advancement in the computer world.
- During the period 1971–1984, Unix underwent further development, was rewritten in the **C language**, and several versions were released, becoming an advanced system capable of meeting general requirements.
- In an attempt by Bell to distribute Unix, it shared it with universities, including **UC Berkeley**, where a free release of Unix was provided. This was the university where developer Ken Thompson had studied.
- In 1975, engineer Ken Thompson returned to UC Berkeley as a professor, bringing with him the latest version of Unix. Joined by two recent graduates, they worked together on tools like the **EX** text editor (which later evolved into the famous **VI** editor) and created custom applications, releasing a software distribution named **BSD** (*Berkeley Software Distribution*) in 1976.
- This prompted Bell Labs to commercialize Unix with a paid license in 1983 to protect its rights. This move angered many programmers and enthusiasts who had contributed to Unix's development over the years, leading several to resign, most notably **Richard Stallman**.

2.2 The Second Stage (1984–1991)

- After resigning, engineer Richard Stallman started developing a project called **GNU** (*GNU's Not UNIX*), aiming to build a free, open-source operating system.
- Thousands of programmers joined the project, writing applications, libraries, shells, and compilers, but they lacked a kernel and relied on the Unix kernel from 1984 through 1991.
- In 1991, a student named **Linus Torvalds** published his graduation project—a kernel compatible with AT-type hard drives that supported multithreading. This research was used to combine the software to form **GNU/Linux**.

> **Note:** The word *Linux* is derived from *Linus* (the kernel developer) and *Tux* (the penguin mascot from the developer's home city).

2.3 The Third Stage (1991–Present)

- In 1993, two major companies emerged—**Slackware** and **Debian**—which took the applications, libraries, and kernel to build their own distribution operating systems.
  - For example, **Debian** developed distributions such as **Ubuntu**, **Chrome OS**, and **Knoppix**.
- In 1994, **Red Hat** appeared, releasing **RHL** in 1998 up to version 9 in 2003, after which it was discontinued.
- In 2003, Red Hat split Linux into two branches:
  - **Fedora**: Offered for free with updates for each release lasting about 13 months.
  - **Red Hat Enterprise Linux (RHEL)**: Aimed at enterprise customers willing to pay for official support.
- Although RHEL builds are not available for free download, GNU/Linux license terms require making the source code available. This allowed the community to recompile the RHEL source code to create free enterprise-grade distributions such as **CentOS** and **AlmaLinux**.

3 Components of Linux System

The Linux OS consists of the following components:

- **Applications / GUI / Utilities / CLI:** These are the programs you interact with, ranging from graphical applications to command-line tools.
- **Shell & System Libraries:** The shell acts as your command interpreter, while libraries provide essential shared resources that applications need to function.

- **System Calls & Services:** This layer provides the interface for applications to communicate with the kernel to request resources or services.
- **Linux Kernel:** The core of the operating system, which handles vital tasks like file management, process scheduling, controlling hardware via drivers, and managing network communication.

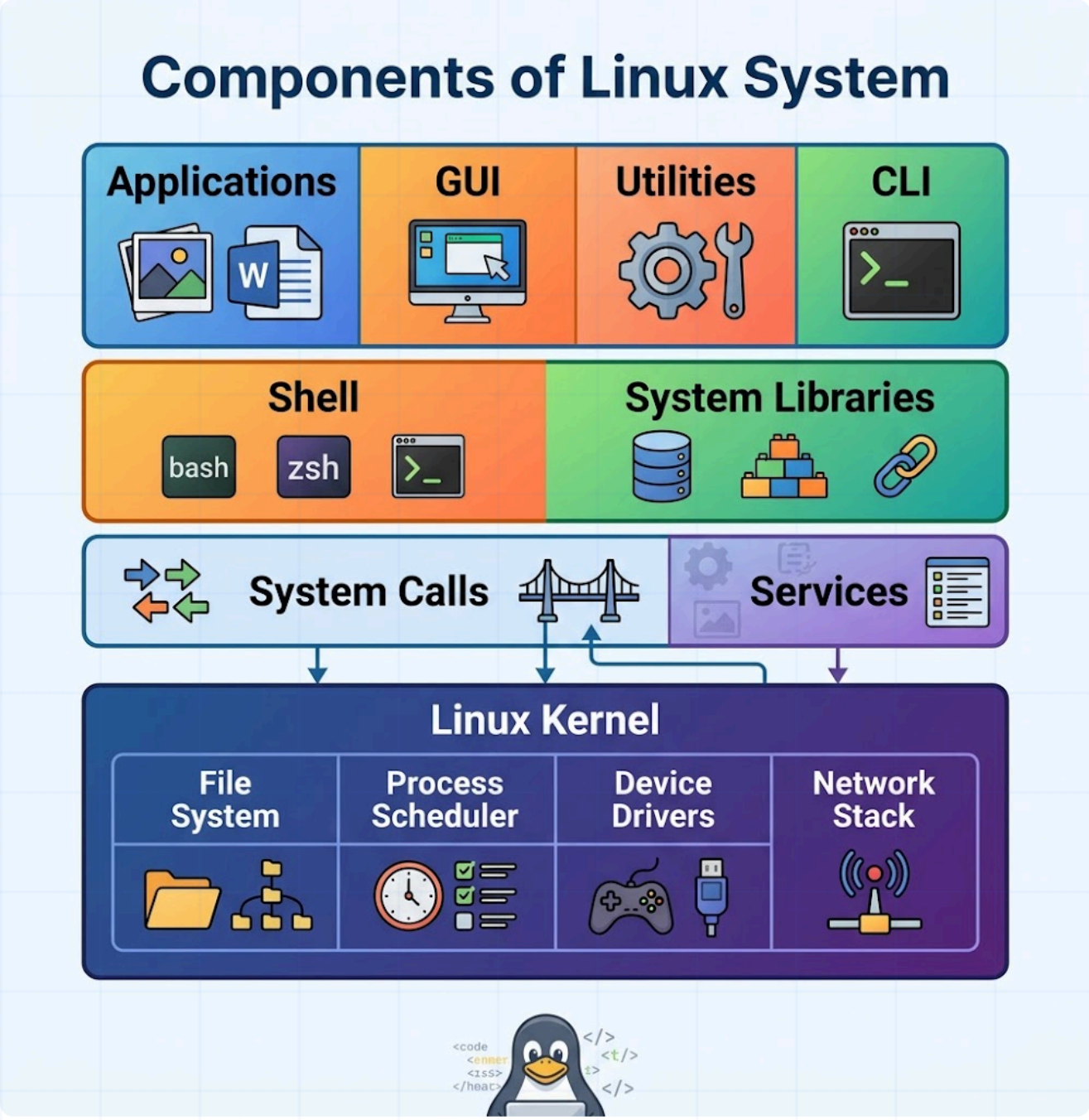


Figure 8: The basic components of a Linux system

3.1 Linux Kernel

The Linux Kernel is the core part of the operating system that establishes communication between hardware and software. For example, the kernel manages memory allocation to run specific programs and manages system resources in general.

It has four main responsibilities:

1. **Device Management:** The system contains many connected hardware devices, such as CPU, memory units, sound cards, graphics cards, etc. The Linux Kernel stores information about all connected devices in Device Drivers. Without the Linux Kernel, it would not be possible to control these devices. The kernel knows how to interact with devices and process their requests to achieve optimal performance, manages communication between all devices, and enforces rules that all devices must follow.
2. **Memory Management:** The kernel tracks used and unused memory and ensures that processes do not interfere with each other's data by using Virtual Memory Addresses.
3. **Processes Management:** A running program is referred to as a process. The kernel acts as an intermediate layer that manages the efficient and fair distribution of system resources (CPU, memory, devices, storage) among multiple concurrent processes. It abstracts physical system resources into dedicated virtual resources for each process, making each process appear as if it is the only one running on the machine, while preventing and mitigating conflicts between processes.
4. **Handling System Calls:** Allows developers to write queries or requests for the kernel to execute specific tasks.

The Linux Kernel consists of the following Core Subsystems:

1. **Process Scheduler:** Controls the fair distribution of CPU time among all processes running concurrently on the system and determines which process gets priority to run.
2. **Memory Management Unit (MMU):** Manages the distribution and allocation of memory resources among different running processes, providing independent virtual address spaces for each process so they do not interfere with one another.

3. **Virtual File System (VFS):** Provides a unified interface to access data stored across different file systems and physical storage media, allowing users and programs to perform operations such as read, write, delete, and modify.

3.2 System Libraries

System libraries act as subprograms that assist in accessing and utilizing kernel features. They provide shared functions and software components to interact with the system and its kernel services. An example is the `Glibc` (GNU C Library), which provides APIs to interact with the system.

3.3 Shell

The Shell acts as a translator between the user and the kernel, providing an interface to input and execute commands on the system. It allows invoking programs, managing storage, and processing data. The most popular shell in Linux is the **Bash Shell** (*Bourne Again Shell*).

The shell lets us interact with Linux via a **Terminal** or **TTY**, allowing execution of binary files to create running processes and interaction with the Linux file system.

3.3.1 Scripting & Commands

- **Scripting:** Allows executing multiple tasks sequentially using files ending with the `.sh` extension.
- **Built-in Commands:** Internal commands integrated directly inside the shell (e.g., `pwd`, `help`, `type`, `set`, `unset`).
- **External Commands (Command Line Utilities):** Programs with their own binary files located in the file system (typically in `/bin`, `/sbin`, `/usr/bin`, `/usr/sbin`) independent of the shell.

3.4 System Concepts & Interaction Interfaces

To effectively navigate and manage a Linux environment, it is essential to understand how the system is deployed, how users interact with command interfaces, and how multiple terminal sessions operate.

3.4.1 Minimal vs. Maximal Deployments

Linux installations typically fall into two main environment configurations depending on their intended use case:

- **Minimal System:** A lightweight installation that operates purely through a Command Line Interface (CLI) without a graphical desktop environment. This setup is standard for servers and cloud instances to minimize resource overhead and maximize performance.
- **Maximal System:** A full-featured installation that includes both a Graphical User Interface (GUI) and a Command Line Interface (CLI). This environment is typically used on personal desktop computers and workstations.

3.4.2 Terminal vs. TTY vs. PTS

While these terms are often used interchangeably in everyday conversation, they refer to distinct technical interface types:

- **Terminal:** In a **Maximal (GUI)** system, a terminal refers to the graphical terminal emulator window (such as GNOME Terminal or Konsole) used to run a shell.
- **TTY (Teletypewriter):** In a **Minimal (non-GUI)** system, the native text interface is called a TTY. Historically derived from physical electromechanical teleprinters, a TTY provides direct physical text access to the machine.
- **PTS (Pseudo Terminal Slave):** A virtual terminal session created when accessing a Linux machine remotely (e.g., via SSH) or inside a graphical terminal emulator.

Note

TTY act as a port (or a communication stream) that sends and receives data to and from the operating system. In Linux and Unix-like operating systems, a TTY acts as a bidirectional communication channel between a user interface (or program) and the kernel.

How TTY Functions as a Data Port

1. "Everything is a File" (Device Files)

In Linux, every TTY interface is represented in the file system as a character device file located under `/dev/`:

- **Physical Console:** `/dev/tty1`, `/dev/tty2`, etc.
- **Pseudo-Terminals (SSH / Terminal Windows):** `/dev/pts/0`, `/dev/pts/1`, etc.

Because it is represented as a device file, the operating system interacts with it using standard input/output stream operations (`read` and `write`):

- **Writing Data (Transmitting):** When a process writes data to `/dev/pts/0`, the operating system routes that data through the TTY driver to be rendered on the screen.
- **Reading Data (Receiving):** When keypresses or data are sent into `/dev/pts/0`, the operating system reads those characters and passes them as `stdin` (standard input) to the running shell or application.

📝 EXAMPLE 1: **Open Ubuntu Terminal in WSL2 & Identify**

Launch **Windows Terminal** from your Start menu and select **Ubuntu** (or type `wsl` in Windows PowerShell).

- Run the `tty` command: `tty`

- **Observation:** The output displays `/dev/pts/2`.
  - **Why?**

✅ Show Solution

Terminal acts as a graphical client attached to a virtual pseudo-terminal slave (`pts`) managed inside WSL2.

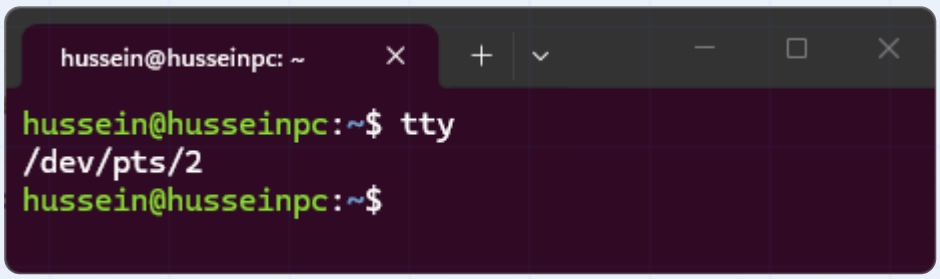


Figure 9: Open Ubuntu Terminal in WSL2 & Identify

📝 EXAMPLE 2: **Open a New Tab and Send Data Between PTS Sessions**

Because WSL2 runs inside a virtualized environment, each new terminal tab or pane creates a distinct **PTS device**.

1. **Open a Second Tab:** Press `Ctrl + Shift + T` (or click the `+` button in Windows Terminal) to open a second Ubuntu tab.
2. In **Tab 2**, run `tty` to check its unique device name.
3. **Send Data from Tab 2 to Tab 1:** In **Tab 2**, write text directly to Tab 1's device file (`/dev/pts/0`):

```bash
echo "Hello from Tab 2!" > /dev/pts/0
```

✅ Show Solution

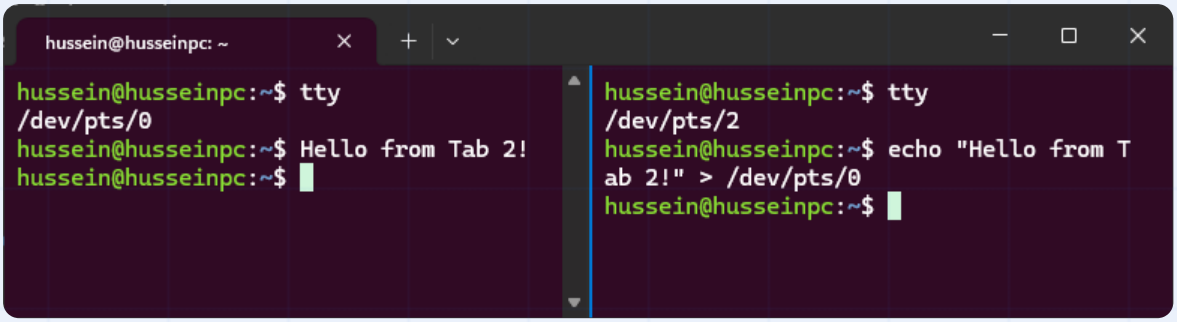


Figure 10: Open a New Tab and Send Data Between PTS Sessions

3.4.3 Virtual TTYs

Linux is a multi-session operating system that supports multiple concurrent local login sessions. These are organized into **Virtual TTYs**:

- **Switching Sessions:** You can switch between different virtual TTYs using the keyboard shortcut `Alt + Ctrl + Fn` (where `n` represents the TTY number, typically ranging from `1` to `6` or `9` depending on the distribution). Alternatively, you can switch sessions using the command `chvt n`.
- **Identifying Your Active TTY:** To check which virtual terminal session you are currently using, execute the `tty` command.
- **Ending a Session:** To log out of the current virtual TTY session, execute the `exit` command.

4 Basic Linux Commands

Before diving into the commands, it is essential to understand the general syntax for Linux commands:

```bash
command [option] [arguments]
```

- `command` : The action or program you want to execute.
- `option` *(optional)*: Used to modify the behavior of the command.
- `arguments` *(optional)*: The targets (files, directories, processes, etc.) that the command applies to.

EXAMPLE 3: clear Command

clear: Clears all previous output from the terminal screen.

```bash
clear
```

EXAMPLE 4: jobs Command

Displays all active background jobs along with their statuses. A job is essentially a process managed directly by the shell.

```bash
jobs
```

EXAMPLE 5: top Command

Equivalent to the Task Manager in Windows. It displays real-time running processes and their CPU and memory usage. It is very useful for monitoring system resources and identifying resource-heavy processes to terminate.

```bash
top
```

✅ Show Result

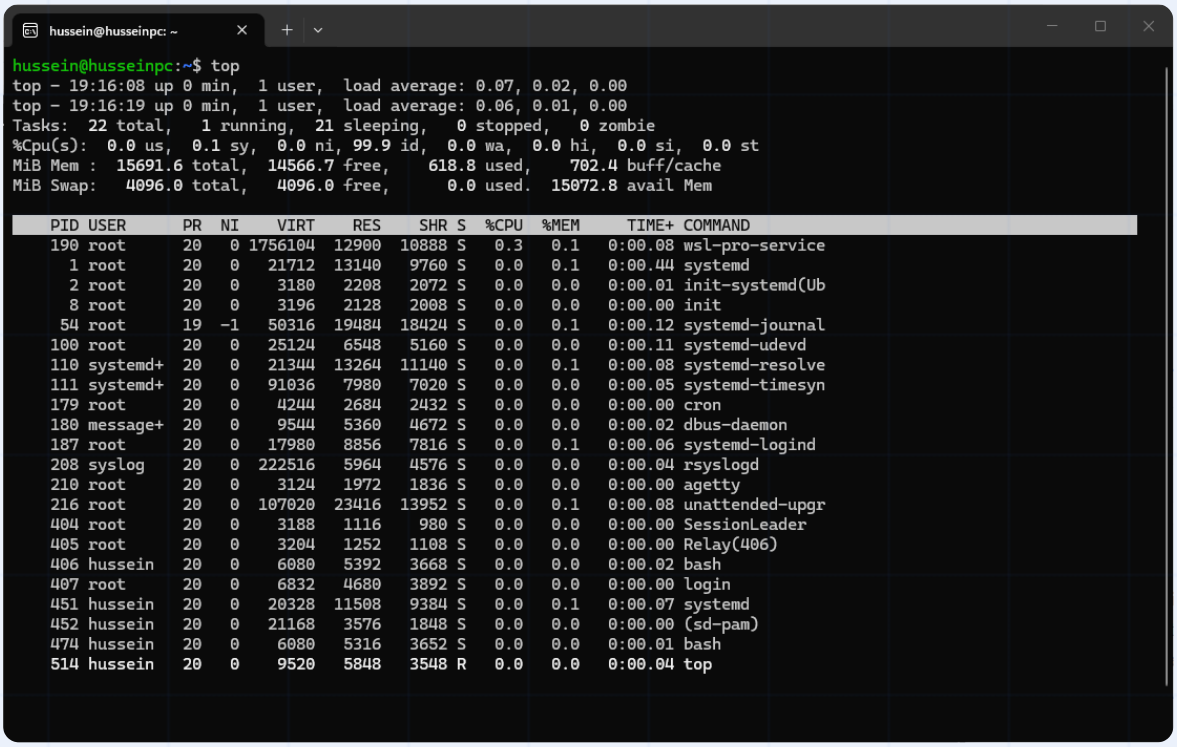


Figure 11: Top Command

📝 EXAMPLE 6: history Command

Lists all previously executed commands in the current shell session, making it easy to review or rerun past commands.

```bash
history
```

✅ Show Result

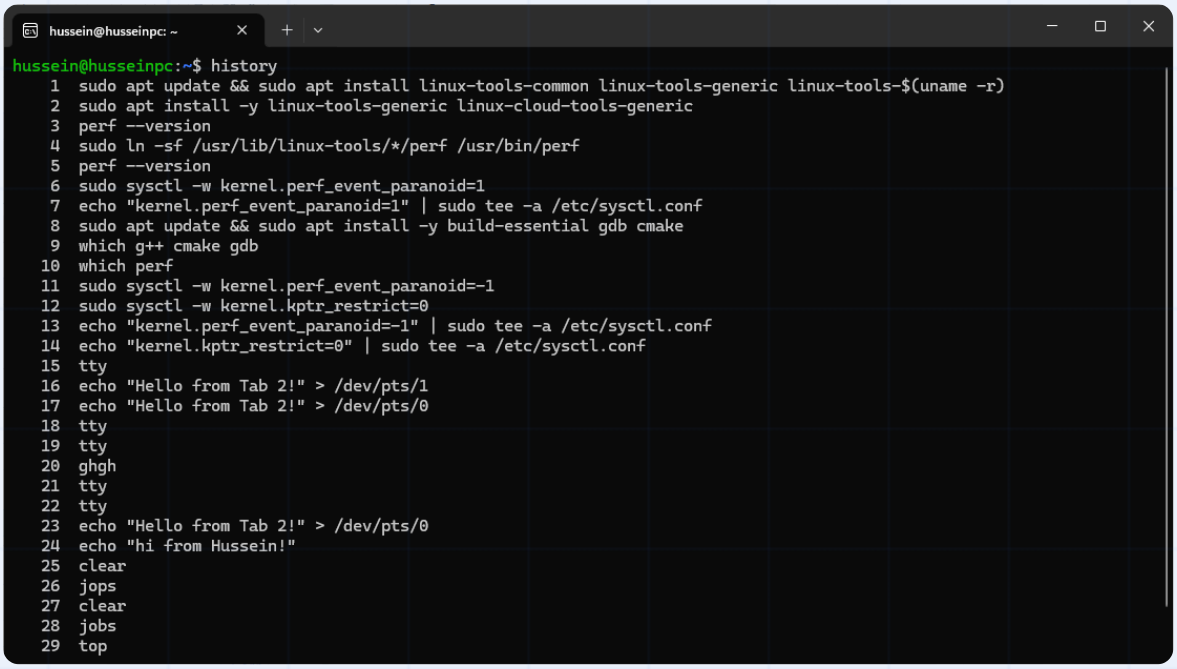


Figure 12: History Command

EXAMPLE 7: man Command

Displays the manual/documentation page for a specific command to explain its usage, options, and parameters.

```bash
man clear
```

EXAMPLE 8: id Command

Displays user identity information, including the User ID (`UID`), Primary Group ID (`GID`), and all Secondary Groups the user belongs to.

```bash
id
```

Show Result

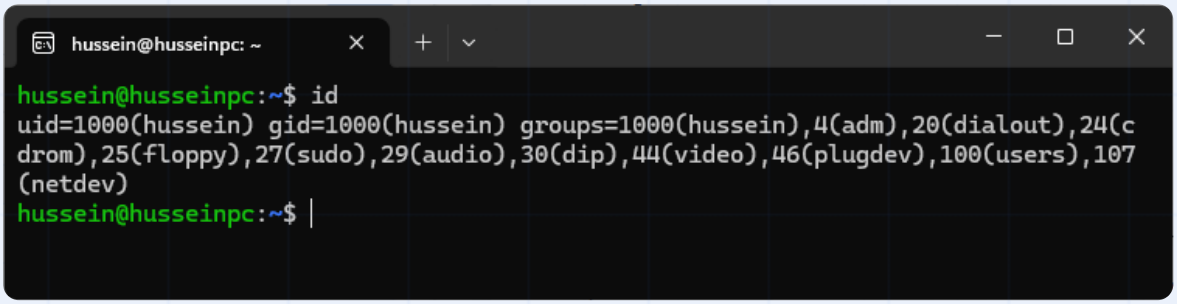


Figure 13: id command

EXAMPLE 9: echo Command

Prints a line of text or variable output directly to the terminal screen.

```bash
echo "Hello, my name is Hussein"
```

Show Result

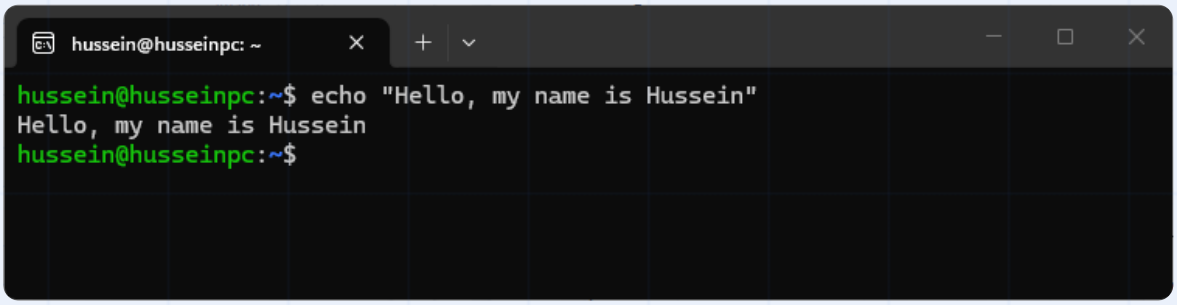


Figure 14: echo command

EXAMPLE 10: hostname Command

Displays the network host name of the system. Adding the `-i` flag displays the system's IP address.

```bash
hostname
hostname -i
```

✅ Show Result

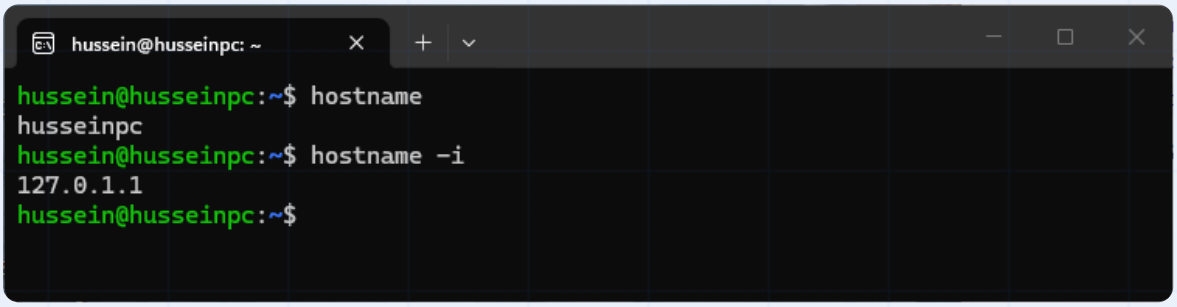


Figure 15: hostname command

5 Use Linux on a Windows Machine

As a developer, you might need to run both Linux and Windows side by side. Luckily, there are several ways you can get the best of both worlds without getting different computers for each operating system.

5.1 Option 1: "Dual-boot" Linux + Windows

With dual boot, you can install Linux alongside Windows on your computer, allowing you to choose which operating system to use at startup. This requires partitioning your hard drive and installing Linux on a separate partition. With this approach, you can only use one operating system at a time.

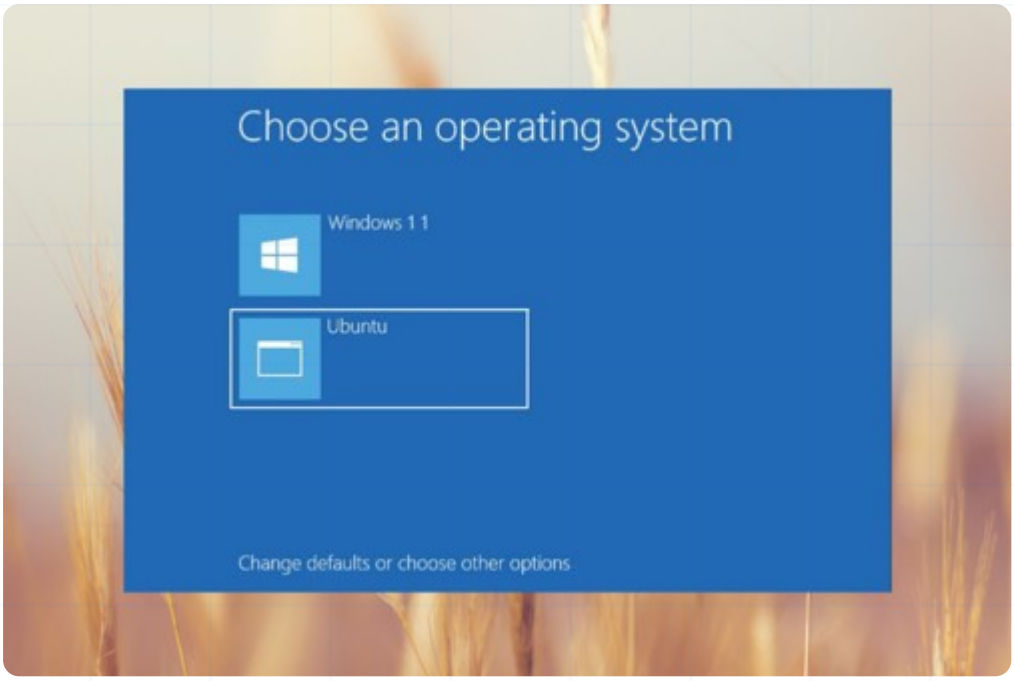


Figure 16: "Dual-boot" Linux + Windows

5.2 Option 2: Use Windows Subsystem for Linux (WSL2)

- Windows Subsystem for Linux provides a compatibility layer that lets you run Linux binary executables natively on Windows.
- The setup for WSL is simple and not time-consuming.
- It is lightweight compared to VMs where you have to allocate resources from the host machine.
- You don't need to install any ISO or virtual disc image for Linux machines which tend to be heavy files.
- You can use Windows and Linux side by side.

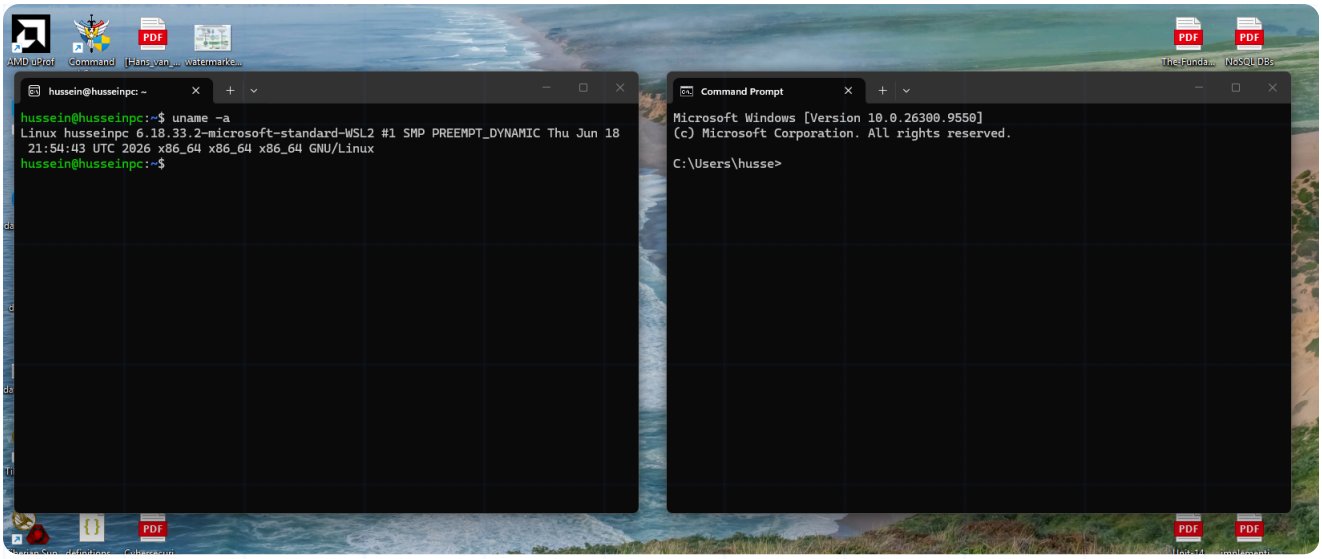


Figure 17: Use Windows Subsystem for Linux (WSL2)



5.3 Option 3: Use a Virtual Machine (VM)

- A virtual machine (VM) is a software emulation of a physical computer system. It allows you to run multiple operating systems and applications on a single physical machine simultaneously.
- VM software provides options to allocate and manage hardware resources for each VM, including CPU cores, memory, disk space, and network bandwidth. You can adjust these allocations based on the requirements of the guest operating systems and applications.

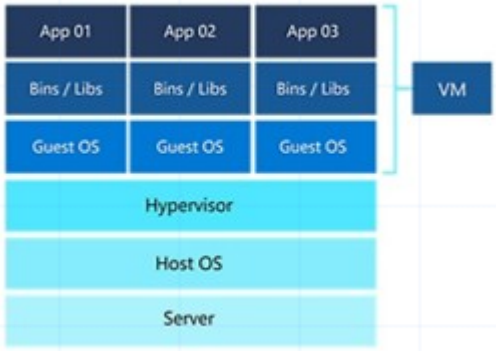


Figure 18: Virtual Machine (VM)

Free Virtual Machine Software

You can use Oracle VirtualBox, which is a popular, free, and open-source platform package released under the GPL version 3 license. It allows you to create and run multiple virtual machines (guest computers) on your host operating system. You can download it directly from the Downloads – Oracle VirtualBox page by choosing your host OS (Windows, macOS, or Linux distributions).

5.3.1 What is Virtualization

DEFINITION 1: **Virtualization**

Virtualization is the process of creating a virtual (rather than actual) version of something, such as hardware platforms, storage devices, or network resources.

> In computing, virtualization uses software—known as a **hypervisor**—to abstract physical hardware. This allows multiple virtual machines (**Guest Computers**) to run independently on a single physical machine (**Host Computer**), sharing its CPU, memory, and storage resources.

A hypervisor is a software layer that allows multiple operating systems to run on a single physical hardware platform. It creates and manages virtual machines (VMs), which are isolated environments that can run
their operating systems independently of each other. The hypervisor abstracts the physical hardware, allocating resources to each VM and ensuring they operate efficiently and securely. There are two main types of hypervisors:

- Type 1 (bare-metal) hypervisors run directly on the host hardware
- Type 2 (hosted) hypervisors run on a conventional operating system like Windows or Linux.

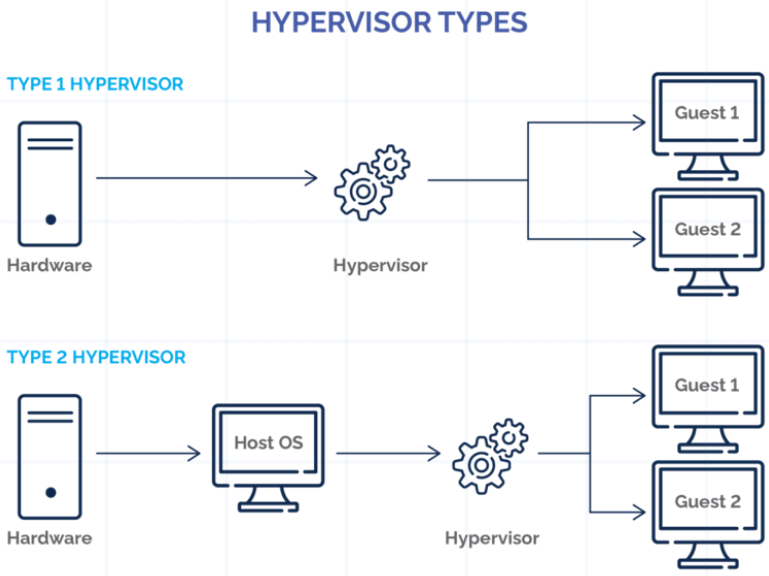


Figure 19: hypervisor Types